Langchain: Agent Systems & Tooling

Архитектура масштабируемых агентных систем в корпоративном контуре.

INFERENCE

Qwen

Local Deployment

VECTOR STORE

Qdrant

High-Performance

TRACING

LangFuse

Self-Hosted

Андрей Носов

Ведущий AI Архитектор, Рафт

- 13 лет в разработке решений на основе NLP и AI
- PhD Communication Sciences, Tampere University
- Руководитель школы AI архитекторов OTUS

@parallelnominded

andrey.nosoff@gmail.com

@AI_ARCHNADZOR

Агенты Langchain

Сравнение архитектур

ХАРАКТЕРИСТИКА	CHAIN (LCEL)	SEMANTIC ROUTER	AGENT (TOOL USE)
Логика управления	ЖЕСТКАЯ Hardcoded sequences	ВЕТВЛЕНИЕ If / Else routing	ЦИКЛИЧЕСКАЯ Cognitive Loop (ReAct)
Предсказуемость	**Высокая (100%)**	**Высокая**	**Средняя** Зависит от качества LLM
Latency	Низкая	**Очень низкая** (<50ms)	**Высокая** Много генераций подряд
Use Case	• Simple RAG • Summarization	• FAQ Routing • Intent Classification	• Сложные многоходовки • Data Analysis

АРХИТЕКТУРНЫЙ ВЫВОД

Используйте **Agents** только там, где требуется адаптивность и неизвестно количество шагов заранее. Для жестких процессов лучше подходит **Chain**.

1. Анатомия Custom Agent Tool

Problem: Standard AgentExecutor

- Hides internal logic ("Black Box" abstraction)
- Hard to handle specific errors (e.g., retries, timeouts)
- Difficult to inspect or modify state mid-execution

Solution: Pure LCEL Loop

- **Explicit Control:** `while` loop managing graph state
- **Intervention:** Inject prompts or modify state at any step
- **Visibility:** Full traceability of State transitions

CONTROL CAPABILITIES

- Conditional Branching
- Timeout Handling
- Human-in-the-Loop
- Smart Retries

2. Tool Calling: Under the Hood

Core Mechanism

1. **Decision Making**
   Models like Qwen-2.5 or GPT-4o analyze tool schemas to decide if an external call is needed.

2. **Special Tokens**
   Uses markers like `<tool_call>` and `</tool_call>` to delimit structured output.

Parsing Errors

When model returns invalid JSON (e.g., missing brackets):

OUTPUTFIXINGPARSER
Heuristic fix. Fast but unreliable.

FEEDBACK LOOP
Return error to LLM context ("You forgot }"). **Preferred.**

3. Tool Design with Pydantic v2

The Schema is the Prompt

What the LLM Sees

Models don't see your python function code. They only see the generated `JSON Schema`. Clear field descriptions are more critical than docstrings.

Constraining Choice

Use `Enum` or `Literal` to prevent hallucinated values. If region must be "us" or "eu", enforce it at the type level.

Free Guardrails

Pydantic validation runs **before** your function executes. This acts as a zero-cost guardrail against bad inputs.

- Automatic type coercion (str → int)
- Regex validation for formats (email, UUID)

4. Memory: Managing State in Long Dialogs

The Problem: Context Bloat

- **Infinite Growth:** Long sessions exceed context windows.
- **OOM Errors:** Out of Memory crashes in production.
- **Hallucinations:** Noise increases with context length ("Lost in the Middle").

Strategy 1: Token Trimming

Using `trim_messages` (LangChain 0.3):

- Keep **System Prompt** locked (pinned).
- Keep last K messages (sliding window).
- Drop oldest **ToolMessages** first.

Strategy 2: Summarization

- **Compression:** Collapse old tool outputs into one sentence.
- *Example:* "Successfully checked DB" instead of raw JSON.
- **Policy:** Keep raw data only for the last 2 steps (for retries).

5. Tool Retrieval (Dynamic Context)

Architecture: Dynamic Context

- **Problem:** Cannot fit 100+ tools into LLM context window.
- **Solution:** Store tool schemas in Vector DB (Qdrant).
- **Flow:** User Query → Embedding → Qdrant → Top-N Tools → LLM.

Pitfall: Missing Related Tools

- **Scenario:** Query "Log me in" finds `auth_login`.
- **Risk:** Agent cannot see `auth_logout` or `reset_password` because they weren't semantically close enough.
- **Consequence:** Agent gets stuck if login fails and it needs to reset credentials.

SOLUTION: TOOLKIT RETRIEVAL

Instead of retrieving single functions, retrieve entire **Toolkits** (groups of related functions).

metadata.toolkit_id | retrieve_by_group

6. Production Pitfall: Argument Hallucinations

The "Validator" Pattern

1. TOOL-LEVEL VALIDATION

Don't just execute. Validate existence first and return a specific, handleable error.

```
if not exists(id): return "ERROR: ID_NOT_FOUND"
```

2. SYSTEM PROMPT GUARDRAILS

Explicitly instruct the agent on how to handle the missing data scenario.

```
"If you do not know the User ID, do NOT guess.
Instead, call search_user_by_name or ask the user."
```

3. FEEDBACK LOOP

When the validator returns `ID_NOT_FOUND`, the agent must see this as a trigger to **search**, not to retry with a new random number.

Symptoms & Risks

The "Stuck" Agent

Agent repeatedly calls `list_files` or similar tools without processing the output or changing parameters.

Token Burn

Rapidly consumes budget/quota with zero progress. Often triggered by "Not Found" errors leading to retry loops.

Prevention Patterns

Hard Limit	`max_iterations=10`
Loop Detection	`Hash(History Window)`
Debouncing	Require new evidence

loop_guard.py

```python
import hashlib
from langchain_core.messages import SystemMessage

def detect_loop(state: list, window: int = 3):
    """
    Prevents agent from getting stuck in repetitive cycles.
    Compares hash of current window vs previous window.
    """
    # 1. Need enough history to compare windows
    if len(state) < window * 2:
        return None

    # 2. Fingerprint recent history (Action + Args)
    current_window = state[-window:]
    previous_window = state[-2*window:-window]

    curr_hash = hashlib.md5(str(current_window).encode()).hexdigest()
    prev_hash = hashlib.md5(str(previous_window).encode()).hexdigest()

    # 3. Check for exact repetition
    if curr_hash == prev_hash:
        # 4. Intervene: Inject System Prompt instead of crashing
        return SystemMessage(
            content="SYSTEM ALERT: You are repeating the same actions. "
                    "Stop immediately and try a different approach."
        )

    return None
```

8. Human-in-the-Loop: Interrupt Pattern

The "Pause & Resume" Mechanism

Not all actions should be autonomous. For sensitive operations (payments, data deletion), the agent must **interrupt** its loop and wait for explicit human approval.

`chain.stream()` → `Checkpoint`

UI/UX IMPLEMENTATION

APPROVAL REQUIRED

ID: trace_8f92a

```
Action: delete_database
Target: "production_db"
```

Approve | Reject | Edit Args

Строгая типизация (Pydantic)

Проблема: Галлюцинации типов

LLM — это текстовые генераторы. Они часто возвращают числа как строки ("20"), путают форматы дат или пропускают обязательные поля, что ломает выполнение кода.

Feedback Loop Architecture

STEP 1: GENERATION

`tool_call(age="twenty")`

STEP 2: VALIDATION ERROR

`ValidationError: value is not a valid integer`

STEP 3: SELF-CORRECTION

`tool_call(age=20)` ✓

schema_validation.py

```python
from pydantic import BaseModel, Field, ValidationError

# 1. Define strict schema
class SearchFilter(BaseModel):
    limit: int = Field(..., description="Max results (1-10)")
    sort_by: str = Field("date", pattern="^(date|relevance)$")

# 2. Simulate LLM Output (String instead of Int)
raw_args = {"limit": "five", "sort_by": "date"}

try:
    # 3. Validate before execution
    params = SearchFilter(**raw_args)
except ValidationError as e:
    # 4. Catch error for feedback loop
    error_msg = f"Argument error: {e}"
    print("System to Agent:", error_msg)
```

RUNTIME OUTPUT

```
> System to Agent: Argument error: 1 validation error for SearchFilter
limit
  value is not a valid integer (type=type_error.integer)
```

10. Routing: Intent Classifier

Why Routing?

Not every query requires a powerful Agent or RAG pipeline. Using a unified chain for "Hello" or simple lookups is inefficient and expensive.

Semantic Router Pattern

- **Zero-Shot Classifier:** Use lightweight models (BERT/DeBERTa) on CPU to classify intent *before* invoking the main LLM.
- **Target Routes:**
  - → **RAG:** "What is the policy on X?"
  - → **Agent:** "Book a meeting for tomorrow."
  - → **Chitchat:** "Hi there!"

Performance Gain

Latency reduction for non-complex queries

-80%

Обработка ошибок в Enterprise

Каскадный Fallback

Стратегия эскалации: Старт с быстрой **Qwen-1.5B**. При ошибке переключение на умную **Qwen-7B**, затем на **Qwen-32B** (через vLLM API).

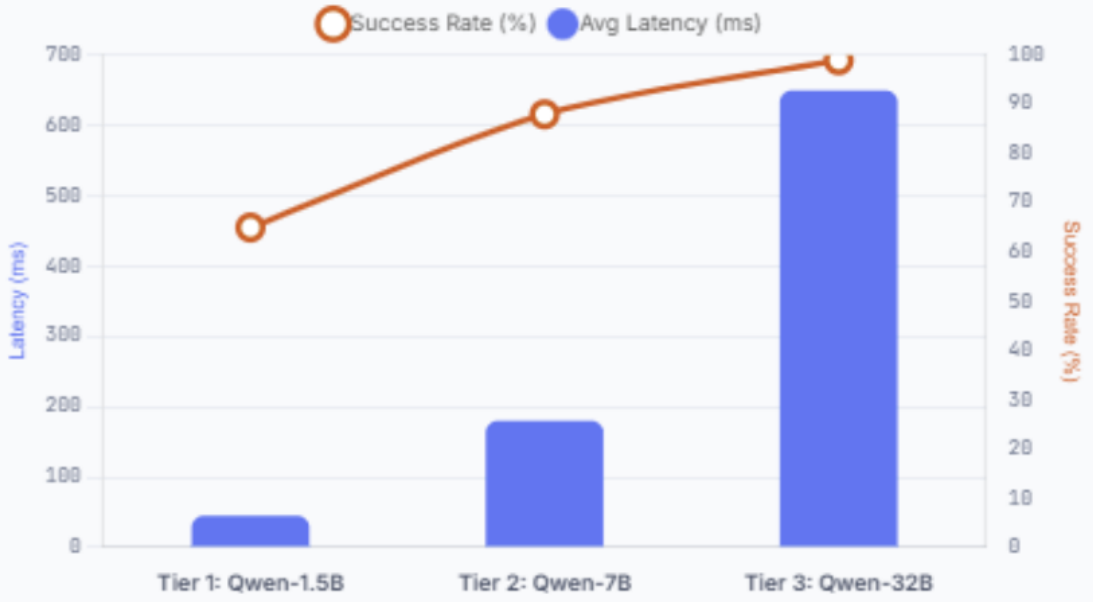
Механизм Retry

Стандарт `max_retries=3`. Используется экспоненциальная задержка (backoff) и передача текста ошибки обратно в контекст модели для самоисправления.

Политики переключения

Триггеры для эскалации: JSONDecodeError (нарушение структуры), галлюцинация имени инструмента, или низкий `logprob` ответа.

EFFICIENCY VS. RELIABILITY TRADE-OFF



** Данные эмуляции нагрузочного тестирования (Latency vs Success Rate)*

12. Security: Code Execution Isolation

The Risk: PythonREPL

Standard tools run in the host process. A malicious prompt (injection) can execute:

```
import os; os.system('rm -rf /')
```

Result: Full server compromise or environment variable exfiltration.

The Solution: Containerization

1. **Ephemeral Sandboxes**

   Execute every code snippet in a fresh, isolated Docker container that is destroyed immediately after execution.

2. **Isolation Policy**

   Enforce strict limits to prevent DoS and data leaks.

   - No Network Access (unless whitelisted)
   - CPU & Memory Quotas
   - Read-only Filesystem

secure_executor.py

```python
import subprocess
import uuid

def execute_in_sandbox(python_code: str) -> str:
    """
    Executes unsafe code inside a locked-down Docker container.
    """
    container_name = f"sandbox_{uuid.uuid4().hex}"

    # Docker command with strict security flags
    cmd = [
        "docker", "run", "--rm",          # Remove container after run
        "--name", container_name,
        "--network", "none",              # 1. NO INTERNET (Data Exfiltration protection)
        "--cpus", "0.5",                  # 2. CPU Quota (DoS protection)
        "--memory", "512m",               # 3. Memory Limit
        "--pids-limit", "50",             # 4. Prevent fork bombs
        "--user", "1000:1000",            # 5. Run as non-root
        "--read-only",                    # 6. Immutable filesystem
        "-v", "/tmp/scratch:/workspace",  # Mount safe scratch space
        "python:3.11-slim-secure",        # Custom hardened image
        "python", "-c", python_code
    ]

    try:
        result = subprocess.run(
            cmd, capture_output=True, text=True, timeout=10 # Hard timeout
        )
        return result.stdout if result.returncode == 0 else result.stderr

    except subprocess.TimeoutExpired:
        subprocess.run(["docker", "kill", container_name])
        return "Error: Execution timed out (Possible infinite loop)."
```

Alternative: For managed infrastructure, consider using **E2B** (e2b.dev) or **gVisor**. These provide VM-level isolation which is safer than standard containers.

Unit Testing Strategy

1 Mocking Tools

Test each tool in isolation. Mock external APIs (e.g., Stripe, SQL) to ensure deterministic tool outputs.

Integration (DeepEval)

GOAL ORIENTED TESTING

Don't just check text similarity. Check if the agent reached the target state (e.g., `FileSaved` state).

METRIC: GOAL COMPLETION RATE

Percentage of runs where agent successfully completes the task within N steps.

CI Pipeline: Run regressions on cost, latency, and quality on every PR.

tests/integration_test.py

```python
from deepeval import assert_test
from deepeval.metrics import AnswerRelevancyMetric
from my_agent import run_agent_graph

def test_agent_goal_completion():
    """
    Integration: Verify agent achieves 'DataSaved' state.
    """
    # 1. Define Goal & Input
    query = "Save user 'Alice' to the database"

    # 2. Run Agent Graph (Mocked DB Connection)
    result = run_agent_graph({"messages": [query]}, mock_db=True)

    # 3. Assert State Transition (Goal Completion)
    # Did the agent call the correct tool?
    tool_calls = [m.tool_calls for m in result['messages'] if m.tool_calls]
    assert any(tc[0]['name'] == 'sql_insert' for tc in tool_calls)

    # 4. DeepEval Quality Check
    # Was the final response relevant to the user?
    metric = AnswerRelevancyMetric(threshold=0.7)
    assert_test(
        input=query,
        actual_output=result['messages'][-1].content,
        metrics=[metric]
    )
```

Best Practice: Use `assert_test` in CI/CD. If the agent starts hallucinating arguments or failing to call tools, the build should fail immediately.

14. Multi-agent Orchestration (Supervisor Lite)

The Supervisor Pattern

Instead of one giant agent with 50 tools, we use a **Controller LLM** to orchestrate specialized sub-agents.

- **Role:** Router & State Manager (Does not call tools directly).
- **Goal:** Selects the next 'Worker' node based on conversation state.

Router Output Schema (JSON)

```
{
  "next_worker": "SQL_Agent" | "Web_Researcher" | "FINISH",
  "instructions": "Extract users joined after 2023..."
}
```

The Supervisor outputs a structured decision mapping to graph nodes via Pydantic.

Worker Contract

- **Isolation:** Workers have their own system prompts and encapsulated tools.
- **Handover:** Workers return updated state + "Done" signal to Supervisor, not a final answer to user.

Наблюдаемость: LangFuse

Self-Hosted Tracing

Задача
Обеспечить полную прозрачность работы агента внутри **корпоративного периметра** без передачи данных во внешние облака.

Agent Run

LangChain Agent выполняет цепочку мыслей и действий

Callback Handler

`on_tool_start, on_llm_end`

Асинхронная отправка трейсов

Docker Container

`langfuse/langfuse`

Внутри периметра компании

Profiling

SQL Tool:	2.5s
LLM Gen:	0.4s
Cost:	$0.00

DOCKER COMPOSE CONFIGURATION

```yaml
services:
  langfuse:
    image: langfuse/langfuse:latest
    ports: "3000:3000"
    environment:
      - DATABASE_URL=postgresql://...
```

```python
# Python Integration
from langfuse.callback import CallbackHandler
handler = CallbackHandler(host="http://localhost:3000")
agent.run(query, callbacks=[handler])
```

TOTAL REQUESTS

14.2k ↑ 12%

AVG LATENCY (P95)

1.8s ↓ 5%

COST / RUN

$0.04 avg

ERROR RATE

2.4% ⚠ Alert > 1%

Trace Waterfall: "Financial Analysis Agent"

ID: tr_99a8b

Input Guardrail	120ms
LLM: Plan Generation (GPT-4)	850ms
! Tool: SQL_Query (Slow)	2.1s
	45ms

0ms 1000ms 2000ms 3000ms

Token Distribution

Input (Context)	75%
Output (Gen)	25%

Automated Alerting Rules

Configured in LangFuse to notify Engineering via Slack/PagerDuty.

Чек-лист безопасности агентов

Enterprise LangChain 2025

Safe Execution Environment

Launch Criteria Status — 75% Ready

Stability & Resilience

- [x] **Global Timeouts**
  Hard limits on all HTTP tools (e.g., 30s) and LLM inference calls.
- [x] **Recursion Limits**
  Max iterations (e.g., 15 steps) enforced to prevent infinite loops.
- [ ] **Circuit Breakers**
  Automatic backoff for flaky 3rd-party APIs.

Security & Guardrails

- [x] **Type Validation (Pydantic)**
  Strict schema validation at all agent input/output boundaries.
- [x] **Data Redaction**
  Sensitive Data Filtering (PII, API keys) in logs and traces.
- [x] **Sandbox Execution**
  Code tools run in isolated containers (Docker/E2B).

Operations & Obs

- [x] **Observability Stack**
  Full tracing (LangFuse/Arize) integrated for all chains.
- [ ] **Alerting Policy**
  Notifications configured for Error Rate > 5% or Latency spikes.
- [ ] **Rollback Strategy**
  Feature flags enabled for gradual rollout; quick revert plan.

План практики

DevOps Automation Scenario

Входящий запрос (User Query)

"Найди простаивающие сервера (CPU < 5%) и выгрузи их список в CSV"

Semantic Router

Классификация интента через Qdrant

Domain: DevOps

Tool Retrieval

Поиск инструментов по вектору

- ✓ get_server_metrics
- ✓ export_to_csv

Agent (Qwen)

Orchestrator

Generates Plan & Calls

Execution

- Metrics fetched
- Data Filtered

Result

Файл сгенерирован

report.csv

AGENT REASONING CHAIN (COT)

```
Thought: The user wants to find idle servers. I should use get_server_metrics first.
Action: get_server_metrics() -> [{"id": 1, "cpu": 2}, {"id": 2, "cpu": 45}]
Observation: Found 2 servers. Only ID=1 matches criteria (CPU < 5%).
Action: export_to_csv(data=[{"id": 1}]) -> "Done"
```

Паттерн «Дата инженер»

Архитектурное решение

Вместо одного "универсального" агента, мы создаем **специализированного субагента**, который имеет доступ только к инструментам анализа данных (Pandas, SQL) и возвращает проверенный результат главному агенту.

User Query

"Посчитай среднюю зарплату в отделе DevOps"

Main Agent

Orchestrator

Решает делегировать

Data Engineer

Pandas | SQL

Sandbox

df['salary'].mean()

Изолированный запуск

Precise Data

5,250

Возврат в Main Agent

DELEGATION IMPLEMENTATION

```
class DataEngineerAgent(Agent):
  tools = [PythonAstREPLTool(locals={"df": df}), SQLDatabaseTool(db=db)]
  system_prompt = "You are a data expert. Output only the final calculated number."

# Main agent logic
if "count" in user_query: return data_engineer.invoke(user_query)
```

Оптимизация памяти

Проблема: CUDA OOM (Out of Memory)

В Colab (T4 GPU 16GB) одновременная загрузка LLM (7B), Embedding-модели и Reranker часто приводит к краху сессии. Python Garbage Collector не освобождает VRAM автоматически.

Pipeline Оптимизации

STEP 1: SEQUENTIAL EXECUTION

Не держим все модели в памяти. Загрузили → Использовали → Выгрузили.

STEP 2: AGGRESSIVE CLEANUP

`del model; gc.collect(); torch.cuda.empty_cache()`

STEP 3: IN-MEMORY INFRASTRUCTURE

Использование Qdrant в RAM режиме (`:memory:`) для ускорения тестов.

gpu_utils.py

```python
import torch
import gc
from qdrant_client import QdrantClient

def flush_memory():
    # 1. Force Python Garbage Collection
    gc.collect()
    # 2. Clear CUDA Cache (Critical for Colab)
    torch.cuda.empty_cache()
    print("INFO: GPU Memory flushed")

# --- Usage in Pipeline ---

# A. Run Heavy Embedding Model
embed_model = load_model("BAAI/bge-m3")
vectors = embed_model.encode(documents)

# B. Unload immediately
del embed_model
flush_memory()  # Frees ~2GB VRAM

# C. Lightweight Vector DB (No Disk I/O)
client = QdrantClient(location=":memory:")
```

TERMINAL OUTPUT (NVIDIA-SMI)

Ресурсы и Литература

VECTOR STORE

Qdrant Documentation

Официальная документация по развертыванию, гибридному поиску и работе с пайлоадами.

qdrant.tech/documentation

ORCHESTRATION

LangChain Agents

Гайды по созданию кастомных инструментов (Custom Tools) и архитектуре агентов (LCEL).

python.langchain.com

OBSERVABILITY

LangFuse Self-Hosted

Инструкция по запуску трассировки локально через Docker Compose без отправки данных вовне.

langfuse.com/self-hosting

github.com/enterprise-rag-2025 | Course Materials PDF

BLOCK 5 COMPLETED

Ваши вопросы

+ если есть вопросы

– если вопросов нет

Практика

Спасибо за внимание!